Fuse Sort Algorithm

A Proposal of Divide & Conquer based Sorting Approach with O(nloglogn) Time and Linear Space Complexity

Yashwant Singh Patel[1], Nitish Kumar Singh[2], Lalit Kumar Vashishtha[3]
Department of Computer Science[1], School of Computer Engineering[23]
CDGI[1], KIIT University[23]
Indore[1], Bhubaneswar[23], India

***Abstract*— Computational Complexity is a fundamental research area in the field of computer science. It has attracted lots of interest of various researchers. In past, vast number of sorting algorithms has been proposed by various researchers. To efficiently optimize any sorting problem having large number of elements requires O(nlogn) time in average case by existing sorting techniques. This paper presents a new sorting technique based on divide & conquer approach, named as Fuse sort algorithm, an approach of comparison based sorting with O(nloglogn) time and linear space. The priory and mathematical analysis of proposed sorting algorithm is given and a case study with merge sort is performed based on several factors.**

Keywords*— *Computational Complexity, Divide & Conquer Approach, Merge Sort Algorithm, Time Complexity, Space Complexity.

I. INTRODUCTION

In modern world, where information is growing rapidly and to search such information there is a need of some proper arrangements. These arrangements are nothing but some special orders like in numerical, lexicographic orders etc. This ordering is provided by sorting algorithms, which is widely used in the field of computer science because of their broad application in areas like for searching methodologies in various search engines, pattern matching, data processing and statistics, digital filters and database systems[1] etc. Since, the starting of algorithmic computation sorting problem has broadly attracted many eminent researchers due to its complexity during solving problem efficiently. The measurement of any sorting algorithm is based on following parameters as discussed in [2, 3] such as

(i) Computational complexity in all the cases such as worst, average and best: If the problem size is n then its best case, the good nature of sorting algorithm is O(n), in average case, the medium nature of sorting algorithm is O(nlogn) and in worst case, the bad nature of sorting algorithm is $O(n^2)$. An ideal sorting algorithm in all the cases takes O(nlogn) time like merge sort.
(ii) Check whether the sorting technique is comparison based or not: An algorithm is said to be comparison based if it is using any comparison operator to compare any two elements otherwise it is said to be non-comparison based.
(iii) Number of swaps: It is identified in the case of in-place algorithms like Quick sort where only single array is used for comparison and within a single array all elements can be sorted.
(iv) Stability: It is used to maintain the relative order of same elements. Any sorting algorithm is stated as stable if it returns the correct order of same elements in output of sorted elements.
(v) Recursive Nature: Some algorithms follow the recursive nature to solve any problem and some follows non recursive nature.
(vi) Memory usage in terms of space: During the procedure of sorting the memory usage is measured in terms of space that is taken by any sorting algorithm. An ideal sorting algorithm takes only O(1) or linear space to sort the any problem size.

There are several sorting methods which are thoroughly studied and widely used such as bubble sort [4], insertion sort, selection sort, merge sort, quick sort[5], heap sort, shell sort[6] etc. as stated in [7]. The complexity analysis of these comparison based sorting algorithms in all the cases with other attributes like stability and In-place is discussed in the Table I.

TABLE I. COMPARISON OF EXISTING SORTING ALGORITHMS

Sorting Algorithm	Best Case	Average Case	Worst Case	Stability	In-Place
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	Yes
Heap Sort	O(nlogn)	O(nlogn)	O(nlogn)	No	Yes
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	Yes
Merge Sort	O(nlogn)	O(nlogn)	O(nlogn)	Yes	No
Quick Sort	O(nlogn)	O(nlogn)	$O(n^2)$	No	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes
Shell Sort	$O(n(\log n)^2)$	$O(n^{3/2})$	O(n)	No	Yes

978-1-4799-4674-7/14/$31.00 ©2014 IEEE

The above discussed algorithms are less useful when problem size is large. In all the cases only merge sort performs better by taking O(nlogn) time complexity. In this paper, a new sorting approach, named as Fuse sort, is presented. It is based on the divide & conquer technique. A mathematical computation based priory analysis is done. The study shows that the proposed algorithm takes O(nloglogn) time complexity and consume linear space in all the cases. Its performance is compared with merge sort algorithm because merge sort is widely used and better to solve large size problems. This paper is arranged as follows in section 2 the approach of divide & conquer is explained on which the proposed algorithm is based. In section 3 the proposed Fuse sort algorithm is described with its mathematical analysis. In section 4 the case study of proposed algorithm with merge sort in various factors is examined. In section 5 the paper is concluded with its future scope.

II. THE APPROACH OF DIVIDE & CONQUER

The Divide & Conquer approach works in following steps-
i) First divide the problem into 2 or more smaller parts known as sub problems until it cannot be further divided.
ii) Solve each sub problem separately.
iii) Combined the sub problems to get the solution of an original problem.

Divide & Conquer approach is widely used to solve difficult problems that are having larger input size problems. Some divide & conquer approaches are Quick sort, Merge Sort etc."

III. PROPOSED FUSE SORT ALGORITHM

The proposed Fuse Sort algorithm is discussed with following points:

A. Basic Concept

It is based on divide & conquer approach. First it divides the n number of elements into $\sqrt{n}$ number of parts. Each part is having $\sqrt{n}$ size. It kept on continue and divide the problem into smaller problems until each sub part minimum size is 2.After dividing the problem into $\sqrt{n}$ halves having size$\sqrt{n}$. It starts to sort each sub part with the help of recursion method. At last each sorted sub-part is combined to form the solution of original problem.

B. Representation

The basic structure of fuse sort algorithm is shown in figure 1 and 2. In figure 1, it is shown that how a problem which is having size n can be divided into $\sqrt{n}$ sub arrays having $\sqrt{n}$ size. While in figure 2, it is shown that each of the divided arrays can be solved recursively to obtain the solution of original problem

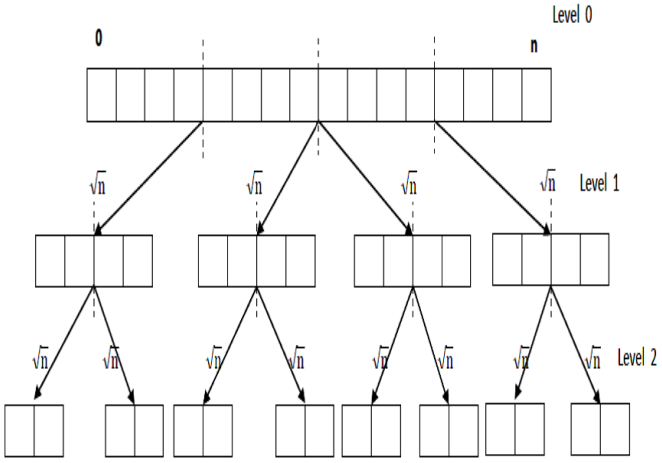


Fig. 1: Dividing the problem size n into $\sqrt{n}$ sub-arrays having $\sqrt{n}$ size

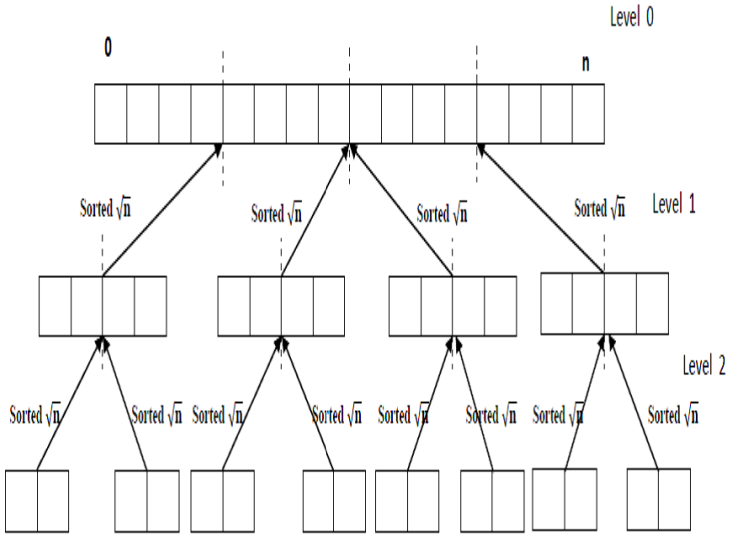


Fig. 2: Solving each array recursively and combining the sorted sub-arrays to obtain the solution of original problem

C. Procedure

The fuse sort algorithm works in following steps:

Step1: Take the n number of elements and divide the problem into $\sqrt{n}$ number of sub problems having size $\sqrt{n}$ and kept on dividing it until each sub problem is not having minimum size 2.

Step2: Sort each sub problem recursively by using fuse sort algorithm.

Step3: After successfully sorting of each sub-problem combine the sorted sub-problems to get the solution of given problem.

D. Pseudo Code For Fuse Sort Algorithm

The pseudo code of proposed Fuse sort algorithm might be expressed as:

Notations: A represents the array

x : index of starting point

z : index of end point

n : Splitting point

i : sub array index a_i a_1 a_2.....

i*n : initial point of the array or end point of the sub array

n*(i+1)-1 : Splitting point of the array or end point of the sub array

j : index value of the array

Fuse_sort (A, x, z)

1). if size[A] < = 2

2). j ← 0

3). if x[j] > x[j+1]

4). exchange x[j] with x[j+1]

5). end if

6). else

7). x[j] ←x[j] // no operation

8). x[j+1] ←x[j+1] // no operation

9). end else

10). end if

11). else

12). n = $\sqrt{z}$

13). for i ← 0 to n-1

14). Fuse_sort (A, n*i, n*(i+1)-1)

15). Fuse (A, x, n*(i+1)-1, z)

16). end for

17). end else

Fuse (A, x, n*(i+1)-1, z)

1). for i = 0 to n-1

2). a_i[ ] = n*(i+1)-1 // sub array size

3). Let a_i[ ] // be new arrays

4). end for

5). for i= 0 to n-1

6). for j = 0 to n-1

7). a_i[j] = A[n*i + j]

8). end inner for loop

9). end outer for loop

10). while (j < = n)

11). i = 0

12). while (i+1 < = n)

13). if (a_i[j]<= a_{i+1}[j])

14). if (S = = null)

15). S ← a_i[j]

16). end if

17). else

18). if (S < = a_i[j])

19). S ← S;

20). end if

21). else

22). S← a_{i+1}[j]

23). end else

24). end else

25). end if

26). else

27). if (S = = null)

28). S← a_{i+1}[j]

29). end if

30). else

31). if (S > = a_{i+1}[j])

32). S← a_{i+1}[j]

33). end if

34). end else

35). end else

36). i←i+1

37). end inner while loop

38). for k = 0 to z

39). A[k] ← S

40). ++ (*(*S))

41). S← null

42). end outer while loop

E. Recurrence Relation of Fuse Sort Algorithm & its Time Complexity Analysis: The type of recurrence relation is polynomial and it is solved by different methods as follows:

1) Substitution Method:

Then complexity of Fuse sort algorithm in the form of recurrence relation can be stated as:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 2 \\ \sqrt{n}\,T(\sqrt{n}) + \Theta(n) & \text{if } n > 2 \end{cases}$$

i) By Substitution Method: This recurrence relation can be solved with the substitution method as:

$T(n) = \sqrt{n}\,T(\sqrt{n}) + \Theta(n)$

$= n^{1/2}\,[n^{1/4}\,T(n^{1/4}) + n^{1/2}] + n$

Repeat up to kth terms

$= n\text{^}(1-(1/2^k))\,T(n\text{^}(1/2^k)) + kn$

$= n/n^{1/2\text{^}k}\,T(n^{1/2\text{^}k}) + kn$

$= n/2\,T(2) + kn$

Where $n^{1/2\text{^}k} = 2$

$(1/2^k)\,\log n = \log 2$ and $\log n = 2^k$

So $k = \log\log n$

$= n + n\log\log n$

$\leq O(n\log\log n)$

2) Recursive Tree Method:

Recursive Tree Method is shown in Figure 3. In which Problem is repeatedly divided into subparts. Finally, the elements obtained in each level are multiplied by height of that tree. Recurrence relation of Fuse Tree is solved below:

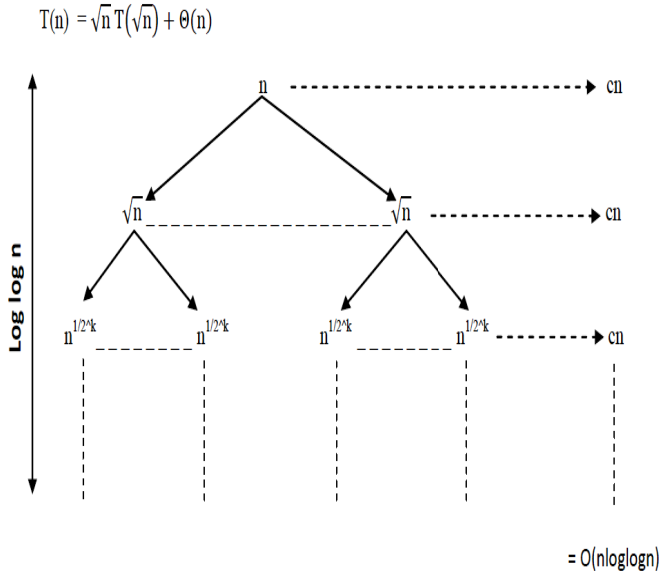


Fig. 3: Recursive Tree Method

The obtained Time complexity of fuse sort is $O(n\log\log n)$. It is nothing but multiplication of two factors one is total height of the Fuse tree which is nothing but $\log\log n$ and the other factor is the number of elements obtained in each level which is cn. So the overall time complexity is $\log\log n$ *cn. It is nothing but $\Theta(n\log\log n)$.

F. Analysis of Fuse Sort Algorithm

The proposed algorithm of Fuse sort will work perfectly when the number of elements will be in power of $2^{2\text{^}i}$ where i = 1, 2, 3, _ _ _ etc. The analysis of recurrence relation can be simplified if it is assumed that the size of original problem is in power of $2^{2\text{^}i}$. In this case, each divide step generates $\sqrt{n}$ sub sequences and each is having size exactly $\sqrt{n}$. This assumption will not affect the order of growth of the recurrence relation.

To build the recurrence for T(n) for the worst case running time analysis of Fuse sort on n number of elements. Fuse sort on two elements takes constant time. If n > 2, in this case running time can be divided as follows:

Divide Time: This step only considers the splitting time of the sub arrays. This is normally constant time. Therefore, D (n) = Θ(1).

Conquer Time: This step recursively solve $\sqrt{n}$ sub problems, each of size $\sqrt{n}$. This adds $\sqrt{n}\,T(\sqrt{n})$ to the running time.

Combine: The whole Fuse procedure on n- element sub-arrays consumes Θ(n) therefore C (n) = Θ(n).

The addition of D(n) and C(n) for the analysis of Fuse sort algorithm will give Θ(n), which is a linear function. Now add it to $\sqrt{n}\,T(n)$ term from the conquer step. It will give the recurrence for the running time of worst case.

T(n) is a running time of fuse sort.

T(n) = Θ(1) if n = 2

$\sqrt{n}\,T(\sqrt{n}) + \Theta(n)$ if n > 2

Let us revise recurrence as

T(n) = c if n = 2

$\sqrt{n}\,T(\sqrt{n}) + \Theta(n)$ if n > 2

In which the constant c represents the time which is required for small size problem having 2 elements in an array at a particular time period. With the help of discussed "Recursive Tree Method" it can be clarified that T(n) is Θ(nlglgn), where lgn stands for $\log_2 n$. Because the growth of logarithm function is much slower than any linear type function. When the input size is large, Fuse sort with its running time Θ(nlglgn) perform better than other sorting algorithm whose worst case running time is $\Theta(n^2)$.

At the point of stability, when some elements are identical then it will maintain the stability and will return there same relative order.

In terms of space it will consume only linear space complexity which is nothing but size of given elements.

IV. CASE STUDY OF FUSE SORT WITH MERGE SORT ALGORITHM

Merge sort is usually known for solving larger problem size. In this case study the merge sort algorithm is compared with fuse sort on the basis of height of tree and number of extra arrays taken to solve the given problem. The case study is shown in Table II.

TABLE II. Case Study of Fuse Sort with Merge Sort algorithm

Number of Elements	**Merge Sort**		**Fuse Sort**	
	Height	**Number of Extra Arrays**	**Height**	**Number of Extra Arrays**
2^2	2	6	1	2
2^4	4	30	2	12
2^8	8	510	3	208
2^{16}	16	131070	4	53504
2^{32}	32	8589934590	5	3506503680
2^{64}	64	$3.689348815*10^{19}$	6	$1.506031863*10^{19}$
2^{128}	128	$6.805647338*10^{38}$	7	$2.778138435*10^{38}$
2^{256}	256	$2.315841785*10^{77}$	8	$9.453515223*10^{76}$

From the above table, it can be observed that Fuse sort is quite efficient compare to Merge sort in case of height and number of arrays to solve any problem size. Height is proportional to access time so fuse sort can also be used for those areas where access time is important. Based on the above analysis the following factors identified as shown in Table III and developed formulas are shown in Table IV.

TABLE III. Some Analyzed Factors

Factors	**Fuse Sort**	**Merge Sort**
Comparison based	Yes	Yes
Length of Smaller Array	2	0 or 1
Stable	Yes	Yes
In-place	No	No
Space Complexity	Linear	Linear
Time Complexity	nloglogn	nlogn

TABLE IV. Some Analyzed Equations

Number of Elements (n)	$2^{2\text{^}i}$ where i = 1,2,3_ _ _
Height of Merge Tree	lg n
Number of extra arrays used for merge sort	$\sum_{i=1}^{2^i} 2\text{^}2^i$
Height of Fuse Tree	i
Number of extra arrays used for fuse sort	$\sum_{i=1}^{2^i}(\sqrt{n}+(\sqrt{n}+(\sqrt{n}*\sqrt{\sqrt{n}}))+(\sqrt{n}+(\sqrt{n}*\sqrt{\sqrt{n}})+(\sqrt{n}+\sqrt{n}*\sqrt{\sqrt{\sqrt{n}}}))+(\sqrt{n}+(\sqrt{n}*\sqrt{\sqrt{n}})+(\sqrt{n}*\sqrt{\sqrt{n}}*\sqrt{\sqrt{\sqrt{n}}})+(\sqrt{n}*\sqrt{\sqrt{n}}*\sqrt{\sqrt{n}+\sqrt{\sqrt{\sqrt{n}}}}))____i$

V. Conculsions And Future Work

In this paper, a new sorting approach is presented. Fuse sort algorithm has O(nloglogn) time complexity in all cases like best, average and worst and it consumes linear space. It takes less number of arrays and height as compare to merge sort algorithm which provides faster access time. The proposed algorithm is compared with merge sort algorithm in terms of time and space complexity. In terms of time it outperforms all the comparison based sorting techniques. To show the efficiency of proposed algorithm a priory analysis based on mathematical computations and a case study with merge sort is performed. Fuse sort will not only perform computations in less time but will also consume linear space to save the memory usage during computations. This approach is very efficient for large problem size which has to perform in less computations and space. So that results can be obtained in less time.

In future, Fuse sort algorithm will be developed with some properties of mathematical relations so that the time and space complexity can be reduced further at some extent.

Acknowledgment

This work was supported by KIIT University, Bhubaneswar, Odisha. We would like to give our sincere thank to our professor Lalit Kumar Vashishtha for his valuable suggestions during various discussion he had made.

References

[1] D.E. Kunth, The Art of Computer Programming: Vol. 3, Sorting and Searching, 2nd printing, Addison-Wesley, Reading, MA, 1975.

[2] Aho A., Hopcroft J., and Ullman J., *The Design and Analysis of Computer Algorithms*, Addison Wesley, 1974.

[3] J.Alnihoud and R.Mansi, "An Enhancement of Major Sorting Algorithms", The International Arab Journal of Information Technology, Vol 7, No.1, January 2010.

[4] Astrachanm O., *Bubble Sort: An Archaeological Algorithmic Analysis*, Duk University, 2003.

[5] *Quick Sort HOARE, C. A.R. Quicksort. Comput. J. 5, i (1962), 10-15.*

[6] Shell Sort Shell D., "A High Speed Sorting Procedure," *Computer Journal of Communications of the ACM*, vol. 2, no. 7, pp. 30-32, 1959.

[7] Cormen T., Leiserson C., Rivest R., and Stein C., *Introduction to Algorithms*, McGraw Hill, 2001